RISC-V Performance Events

Authors: Author 1, Author 2

Version v0.1.0, 2024-09-27: Draft

Table of Contents

Preamble

This document is in the Development state

Expect potential changes. This draft specification is likely to evolve before it is accepted as a standard. Implementations based on this draft may not conform to the future standard.

Copyright and license information

This specification is licensed under the Creative Commons Attribution 4.0 International License (CC-BY 4.0). The full license text is available at creativecommons.org/licenses/by/4.0/.

Copyright 2024 by RISC-V International.

Contributors

This RISC-V specification has been contributed to directly or indirectly by:

- Author1 <required1@email.com>
- Author2 <required2@email.com>

Chapter 1. Introduction

The RISC-V hardware performance monitoring counters (Zihpm) provide support for counting programmable performance events. Such events can provide insights into software execution behavior, insights that are critical when tuning a profiled workload. However, no performance events are standardized, which means profiling tools must comprehend a custom set of events specific to each hardware implementation. This prevents profiling tools from offering general-purpose, event-based analysis capabilities that can be employed regardless of the underlying hardware implementation.

The Performance Events non-ISA extension provides a set of standard performance events and metrics (or formulas of events). For each standard event, the name and the precise hardware behavior associated with it is specified. For each standard metric, the name, precise description, and event formula, including the names of the constituent events, is defined.

This extension does not standardize event selector values - these are left up to implementations.

Chapter 2. Groups

All the events and metrics are split between several groups which are described in details sub-sections below. In addition most of the groups are further split into 2 variants:

- RETIRED - for events counted at retirement. For example, CACHE_RETIRED.L2.LOAD.MISS event will count L2 misses caused by retired load instructions
- SPEC - for speculative events. For example, CACHE_SPEC.L2.LOAD.MISS event will count L2 misses caused by load instructions regardless of whether they were retired or not.

In general RETIRED events look more useful for performance analysis. In addition in the future it may be possible to provide more context for them - e.g. precise sample IP. But on the other hand they it may be significantly more expensive to implement. It is up to implementations to decide if they want to provide RETIRED, SPEC or both variants of a group.

2.1. GEN

This group contains general events not specific to a particular part of CPU pipeline.

Name	Description
CYCLES.HART	Cycles when the hart is not in halt state. The cycles are counted with real (potentially variable) frequency the hart is working at

Table 1. GEN group events

2.2. RETIRED

This group contains events measured at retirement which didn't fall into any specific group below.

Name	Description
INST.RET	Number of instructions retired
INST.LOAD.RET	Number of memory load instructions retired
INST.LOAD.UC.RET	Number of memory load instructions retired which accessed uncacheble memory
INST.STORE.RET	Number of memory store instructions retired
INST.INT.RET	Number of integer instructions retired
INST.FPU.RET	Number of FPU instructions retired
UOP.RET	Number of micro-operations retired

Table 2. RETIRED group events

2.3. SPEC

This group contains speculative events which didn't fall into any specific group below.

Name	Description
INST.SPEC	Number of instructions issued
INST.LOAD.SPEC	Number of memory load instructions issued
INST.STORE.SPEC	Number of memory store instructions issued
INST.INT.SPEC	Number of integer instructions issued
INST.FPU.SPEC	Number of FPU instructions issued
UOP.SPEC	Number of micro-operations issued

Table 3. SPEC group events

2.4. CTRL_FLOW (retirement)

Retirement control flow group which contains events and metrics for measuring things branch mispredictions, data mis-speculations etc.

Name	Description
CTRL_FLOW.RET	Control flow instructions retired
CTRL_FLOW.MISPRED.RET	Control flow instructions mis-predicted
CTRL_FLOW.BRANCH.RET	Conditional branches retired
CTRL_FLOW.BRANCH.MISPRED.RET	Conditional branches mis-predicted
CTRL_FLOW.IND_CALL.RET	Indirect calls retired. Indirect calls are defined using following encodings: 'JALR x1, rs where rs != x5'; 'JALR x5, rs where rs != x1'; 'C.JALR rs1 where rs1 != x5'
CTRL_FLOW.IND_CALL.MISPRED.RET	Indirect calls mis-predicted. Indirect calls are defined using following encodings: 'JALR x1, rs where rs != x5'; 'JALR x5, rs where rs != x1'; 'C.JALR rs1 where rs1 != x5'
CTRL_FLOW.DIR_CALL.RET	Direct calls retired. Direct calls are defined using following encodings: 'JAL x1'; 'JAL x5'; 'C.JAL'
CTRL_FLOW.DIR_CALL.MISPRED.RET	Direct calls mis-predicted. Direct calls are defined using following encodings: 'JAL x1'; 'JAL x5'; 'C.JAL'
CTRL_FLOW.IND_JUMP.RET	Indirect jumps (without linkage) retired. Indirect jumps (without linkage) are defined using following encodings: 'JALR x0, rs where rs != (x1 or x5)'; 'C.JR rs1 where rs1 != (x1 or x5)'
CTRL_FLOW.IND_JUMP.MISPRED.RET	Indirect jumps (without linkage) mis-predicted. Indirect jumps (without linkage) are defined using following encodings: 'JALR x0, rs where rs != (x1 or x5)'; 'C.JR rs1 where rs1 != (x1 or x5)'
CTRL_FLOW.DIR_JUMP.RET	Direct jumps (without linkage) retired. Direct jumps (without linkage) are defined using following encodings: 'JAL x0'; 'C.J'
CTRL_FLOW.DIR_JUMP.MISPRED.RET	Direct jumps (without linkage) mis-predicted. Direct jumps (without linkage) are defined using following encodings: 'JAL x0'; 'C.J'
CTRL_FLOW.COR_SWAP.RET	Co-routine swaps retired. Co-routine swaps are defined using following encodings: 'JALR x1, x5'; 'JALR x5, x1'; 'C.JALR x5'
CTRL_FLOW.COR_SWAP.MISPRED.RET	Co-routine swaps mis-predicted. Co-routine swaps are defined using following encodings: 'JALR x1, x5'; 'JALR x5, x1'; 'C.JALR x5'
CTRL_FLOW.RETURN.RET	Function returns retired. Function returns are defined using following encodings: 'JALR rd, rs where rs == (x1 or x5) and rd != (x1 or x5)'; 'C.JR rs1 where rs1 == (x1 or x5)'
CTRL_FLOW.RETURN.MISPRED.RET	Function returns mis-predicted. Function returns are defined using following encodings: 'JALR rd, rs where rs == (x1 or x5) and rd != (x1 or x5)'; 'C.JR rs1 where rs1 == (x1 or x5)'
CTRL_FLOW.IND_JUMP_LINKAGE.RET	Other indirect jumps (with linkage) retired. Other indirect jump (with linkage) are defined using following encodings: 'JALR rd, rs where rs != (x1 or x5) and rd != (x0, x1, or x5)'
CTRL_FLOW.IND_JUMP_LINKAGE.MISPRED.RET	Other indirect jumps (with linkage) mis-predicted. Other indirect jumps (with linkage) are defined using following encodings: 'JALR rd, rs where rs != (x1 or x5) and rd != (x0, x1, or x5)'
CTRL_FLOW.DIR_JUMP_LINKAGE.RET	Other direct jumps (with linkage) retired. Other direct jump (with linkage) are defined using following encodings: 'JAL rd where rd != (x0, x1, or x5)'
CTRL_FLOW.DIR_JUMP_LINKAGE.MISPRED.RET	Other direct jumps (with linkage) mis-predicted. Other direct jumps (with linkage) are defined using following encodings: 'JAL rd where rd != (x0, x1, or x5)'

Table 4. CTRL_FLOW group events

Name	Description	Formula
CTRL_FLOW.PKI	The rate of control flow instructions retired per kilo instructions	CTRL_FLOW.RET / INST.RET * 1000
CTRL_FLOW.MPKI	The rate of control flow instructions mis-predicted per kilo instructions	CTRL_FLOW.MISPRED.RET / INST.RET * 1000
CTRL_FLOW.MISPRED.RATE	The rate of control flow instructions mis-predicted to the overall predicted control flow instructions	CTRL_FLOW.MISPRED.RET / (CTRL_FLOW.RET - CTRL_FLOW.DIR_CALL.RET - CTRL_FLOW.DIR_JUMP.RET - CTRL_FLOW.DIR_JUMP_LINKAGE.RET)
CTRL_FLOW.BRANCH.PKI	The rate of conditional branches retired per kilo instructions	CTRL_FLOW.BRANCH.RET / INST.RET * 1000
CTRL_FLOW.BRANCH.MPKI	The rate of conditional branches mis-predicted per kilo instructions	CTRL_FLOW.BRANCH.MISPRED.RET / INST.RET * 1000
CTRL_FLOW.BRANCH.MISPRED.RATE	The rate of conditional branches mis-predicted to the overall conditional branches	CTRL_FLOW.BRANCH.MISPRED.RET / CTRL_FLOW.BRANCH.RET
CTRL_FLOW.IND_CALL.PKI	The rate of indirect calls retired per kilo instructions	CTRL_FLOW.IND_CALL.RET / INST.RET * 1000

Name	Description	Formula
CTRL_FLOW.IND_CALL.MPKI	The rate of indirect calls mis-predicted per kilo instructions	CTRL_FLOW.IND_CALL.MISPRED.RET / INST.RET * 1000
CTRL_FLOW.IND_CALL.MISPRED_RATE	The rate of indirect calls mis-predicted to the overall indirect calls	CTRL_FLOW.IND_CALL.MISPRED.RET / CTRL_FLOW.IND_CALL.RET
CTRL_FLOW.DIR_CALL.PKI	The rate of direct calls retired per kilo instructions	CTRL_FLOW.DIR_CALL.RET / INST.RET * 1000
CTRL_FLOW.DIR_CALL.MPKI	The rate of direct calls mis-predicted per kilo instructions	CTRL_FLOW.DIR_CALL.MISPRED.RET / INST.RET * 1000
CTRL_FLOW.DIR_CALL.MISPRED.RATE	The rate of direct calls mis-predicted to the overall direct calls	CTRL_FLOW.DIR_CALL.MISPRED.RET / CTRL_FLOW.DIR_CALL.RET
CTRL_FLOW.IND_JUMP.PKI	The rate of indirect jumps retired per kilo instructions	CTRL_FLOW.IND_JUMP.RET / INST.RET * 1000
CTRL_FLOW.IND_JUMP.MPKI	The rate of indirect jumps mis-predicted per kilo instructions	CTRL_FLOW.IND_JUMP.MISPRED.RET / INST.RET * 1000
CTRL_FLOW.IND_JUMP.MISPRED.RATE	The rate of indirect jumps mis-predicted to the overall indirect jumps	CTRL_FLOW.IND_JUMP.MISPRED.RET / CTRL_FLOW.IND_JUMP.RET
CTRL_FLOW.DIR_JUMP.PKI	The rate of direct jumps retired per kilo instructions	CTRL_FLOW.DIR_JUMP.RET / INST.RET * 1000
CTRL_FLOW.DIR_JUMP.MPKI	The rate of direct jumps mis-predicted per kilo instructions	CTRL_FLOW.DIR_JUMP.MISPRED.RET / INST.RET * 1000
CTRL_FLOW.DIR_JUMP.MISPRED.RATE	The rate of direct jumps mis-predicted to the overall indirect jumps	CTRL_FLOW.DIR_JUMP.MISPRED.RET / CTRL_FLOW.DIR_JUMP.RET
CTRL_FLOW.CORE_SWAP.PKI	The rate of co-routine swaps retired per kilo instructions	CTRL_FLOW.COR_SWAP.RET / INST.RET * 1000
CTRL_FLOW.COR_SWAP.MPKI	The rate of co-routine swaps mis-predicted per kilo instructions	CTRL_FLOW.COR_SWAP.MISPRED.RET / INST.RET * 1000
CTRL_FLOW.COR_SWAP.MISPRED.RATE	The rate of co-routine swaps mis-predicted to the overall indirect jumps	CTRL_FLOW.COR_SWAP.MISPRED.RET / CTRL_FLOW.COR_SWAP.RET
CTRL_FLOW.RETURN.PKI	The rate of function returns retired per kilo instructions	CTRL_FLOW.RETURN.RET / INST.RET * 1000
CTRL_FLOW.RETURN.MPKI	The rate of function returns mis-predicted per kilo instructions	CTRL_FLOW.RETURN.MISPRED.RET / INST.RET * 1000
CTRL_FLOW.RETURN.MISPRED.RATE	The rate of function returns mis-predicted to the overall function returns	CTRL_FLOW.RETURN.MISPRED.RET / CTRL_FLOW.RETURN.RET
CTRL_FLOW.IND_JUMP_LINKAGE.PKI	The rate of indirect jumps (with linkage) retired per kilo instructions	CTRL_FLOW.IND_JUMP_LINKAGE.RET / INST.RET * 1000
CTRL_FLOW.IND_JUMP_LINKAGE.MPKI	The rate of indirect jumps (with linkage) mis-predicted per kilo instructions	CTRL_FLOW.IND_JUMP_LINKAGE.MISPRED.RET / INST.RET * 1000
CTRL_FLOW.IND_JUMP_LINKAGE.MISPRED.RATE	The rate of indirect jumps (with linkage) mis-predicted to the overall indirect jumps (with linkage)	CTRL_FLOW.IND_JUMP_LINKAGE.MISPRED.RET / CTRL_FLOW.IND_JUMP_LINKAGE.RET
CTRL_FLOW.DIR_JUMP_LINKAGE.PKI	The rate of direct jumps (with linkage) retired per kilo instructions	CTRL_FLOW.DIR_JUMP_LINKAGE.RET / INST.RET * 1000
CTRL_FLOW.DIR_JUMP_LINKAGE.MPKI	The rate of direct jumps (with linkage) mis-predicted per kilo instructions	CTRL_FLOW.DIR_JUMP_LINKAGE.MISPRED.RET / INST.RET * 1000
CTRL_FLOW.DIR_JUMP_LINKAGE.MISPRED.RATE	The rate of direct jumps (with linkage) mis-predicted to the overall direct jumps (with linkage)	CTRL_FLOW.DIR_JUMP_LINKAGE.MISPRED.RET / CTRL_FLOW.DIR_JUMP_LINKAGE.RET

Table 5. CTRL_FLOW group metrics

2.5. CTRL_FLOW (speculative)

Speculation prediction group. Unlike most of the groups below prediction events mostly naturally counted only at retirement time. So this group contains only a few events which make sense to count speculatively.

Name	Description
FLUSH.SPEC	Counts pipeline flushes due to all reasons - such as branch misprediction, memory disambiguation, serializing instructions
FLUSH.CYCLES	Cycles to recover from pipeline flushes due to any reason. Examples: branch misprediction, memory disambiguation, serializing instruction

Table 6. CTRL_FLOW group events

2.6. CACHE (retirement)

This group contains events and metrics for data and instruction caches (all levels) counted at retirement.

Name	Description
CACHE.L1D.LOAD.ACCESS.RET	Retired load instruction which accessed L1D cache
CACHE.L1D.LOAD.MISS.RET	Retired load instruction which missed L1D cache
CACHE.L1D.LOAD.HIT.RET	Retired load instruction which hit L1D cache
CACHE.L1D.LOAD.MERGE.RET	Retired load instruction which hit L1D cache with data not yet in cache but was already requested by preceding miss
CACHE.L1D.STORE.ACCESS.RET	Retired store instruction which accessed L1D cache
CACHE.L1D.STORE.MISS.RET	Retired store instruction which missed L1D cache
CACHE.L1D.STORE.HIT.RET	Retired store instruction which hit L1D cache
CACHE.L1D.STORE.MERGE.RET	Retired store instruction which hit L1D cache with data not yet in cache but was already requested by preceding miss
CACHE.L1I.MISS.RET	Retired instruction with L1 Instruction cache miss on fetching
CACHE.L2.LOAD.ACCESS.RET	Retired load instruction which got data from L2 or from some next level in memory hierarchy - L3 cache, local mmemory, remote cache, remote memory, etc.
CACHE.L2.LOAD.MISS.RET	Retired load instruction which got data from some next level (relative to L2) in memory hierarchy - L3 cache, local mmemory, remote cache, remote memory, etc.
CACHE.L2.LOAD.HIT.RET	Retired load instruction which got data from L2 cache
CACHE.L2.LOAD.MERGE.RET	Retired load instruction which hit L2 cache with data not yet in cache but was already requested by preceding miss
CACHE.L3.LOAD.ACCESS.RET	Retired load instruction which got data from L3 cache or from some next level in memory hierarchy - local mmemory, remote cache, remote memory, etc.
CACHE.L3.LOAD.MISS.RET	Retired load instruction which got data from some next level (relative to L3) in memory hierarchy - local mmemory, remote cache, remote memory, etc.
CACHE.L3.LOAD.HIT.RET	Retired load instruction which got data from L3 cache
CACHE.L3.LOAD.MERGE.RET	Retired load instruction which hit L3 cache with data not yet in cache but was already requested by preceding miss
CACHE.LOAD.LOCAL_MEMORY.RET	Retired load instruction which got data from local memory.
CACHE.LOAD.REMOTE_MEMORY.RET	Retired load instruction which got data from remote memory (memory attached to remote socket).
CACHE.LOAD.REMOTE_CACHE.RET	Retired load instruction which got data from remote cache (cache on remote socket).

Table 7. CACHE group events

Name	Description	Formula
CACHE.L1D.LOAD.RET.MPKI	The rate of retired L1 data load cache misses per kilo instructions retired	CACHE.L1D.LOAD.MISS.RET / INST.RET * 1000
CACHE.L1D.LOAD.MISS.RET.RATE	The ratio of retired L1D cache load misses to the total number of retired L1D load accesses	CACHE.L1D.LOAD.MISS.RET / CACHE.L1D.LOAD.ACCESS.RET
CACHE.L1D.LOAD.MERGE.RET.PKI	The rate of retired L1 data load cache misses which merged with previous cache miss per kilo instructions retired	CACHE.L1D.LOAD.MERGE.RET / INST.RET * 1000
CACHE.L1I.RET.MPKI	The rate of retired instructions with L1 instruction cache misses per kilo instructions retired	CACHE.L1I.MISS.RET / INST.RET * 1000
CACHE.L1D.STORE.RET.MPKI	The rate of retired L1 data store cache misses per kilo instructions retired	CACHE.L1D.STORE.MISS.RET / INST.RET * 1000
CACHE.L1D.STORE.MISS.RET.RATE	The ratio of retired L1D cache store misses to the total number of retired L1D store accesses	CACHE.L1D.STORE.MISS.RET / CACHE.L1D.STORE.ACCESS.RET
CACHE.L2.LOAD.RET.MPKI	The rate of retired L2 data load cache misses per kilo instructions retired	CACHE.L2.LOAD.MISS.RET / INST.RET * 1000
CACHE.L2.LOAD.MISS.RET.RATE	The ratio of retired L2 cache load misses to the total number of retired L2 load accesses	CACHE.L2.LOAD.MISS.RET / CACHE.L2.LOAD.ACCESS.RET
CACHE.L3.LOAD.RET.MPKI	The rate of retired L3 data load cache misses per kilo instructions retired	CACHE.L3.LOAD.MISS.RET / INST.RET * 1000
CACHE.L3.LOAD.MISS.RET.RATE	The ratio of retired L3 cache load misses to the total number of retired L3 load accesses	CACHE.L3.LOAD.MISS.RET / CACHE.L3.LOAD.ACCESS.RET

Table 8. CACHE group metrics

2.7. CACHE (speculative)

This group contains events and metrics for data and instruction caches (all levels) counted speculatively.

Name	Description
CACHE.L1D.LOAD.ACCESS.SPEC	L1D cache accesses for load instructions. Speculatively executed instructions are also taken into account.
CACHE.L1D.LOAD.MISS.SPEC	L1D cache misses for load instructions. Speculatively executed instructions are also taken into account.
CACHE.L1D.LOAD.HIT.SPEC	L1D cache hits for load instructions. Speculatively executed instructions are also taken into account.
CACHE.L1D.LOAD.MERGE.SPEC	L1D cache hits for load instructions where data is not yet in cache but was already requested by preceding miss. Speculatively executed instructions are also taken into account.
CACHE.L1D.LOAD.MISS_OUTSTANDING.CYCLES	Cycles while at least one load L1 data cache miss in progress.
CACHE.L1D.STORE.ACCESS.SPEC	L1D cache accesses for store instructions. Speculatively executed instructions are also taken into account.
CACHE.L1D.STORE.MISS.SPEC	L1D cache misses for store instructions. Speculatively executed instructions are also taken into account.
CACHE.L1D.STORE.HIT.SPEC	L1D cache hits for store instructions. Speculatively executed instructions are also taken into account.
CACHE.L1D.STORE.MERGE.SPEC	L1D cache hits for store instructions where data is not yet in cache but was already requested by preceding miss. Speculatively executed instructions are also taken into account.
CACHE.L1D.PREF.ISSUED	Prefetcher requests issued by L1D to next level cache.
CACHE.L1D.PREF.UNUSED	Number of cachelines brought into L1D by prefetcher and evicted without being accessed even once.
CACHE.L1D.WB	Writebacks from L1D to next level cache or memory.
CACHE.L1I.ACCESS.SPEC	L1I cache accesses.
CACHE.L1I.MISS.SPEC	L1I cache misses.
CACHE.L1I.HIT.SPEC	L1I cache hits.
CACHE.L1I.MERGE.SPEC	L1I cache hits data is not yet in cache but was already requested by preceding miss.
CACHE.L1I.MISS_OUTSTANDING.CYCLES	Cycles with L1 Instruction cache miss in progress.
CACHE.L2.LOAD.ACCESS.SPEC	L2 cache accesses initiated by load instructions. Speculatively executed instructions are also taken into account.
CACHE.L2.LOAD.MISS.SPEC	L2 cache misses initiated by load instructions. Speculatively executed instructions are also taken into account.
CACHE.L2.LOAD.HIT.SPEC	L2 cache hits initiated by load instructions. Speculatively executed instructions are also taken into account.
CACHE.L2.LOAD.MERGE.SPEC	L2 cache hits initiated by load instructions where data is not yet in cache but was already requested by preceding miss. Speculatively executed instructions are also taken into account.
CACHE.L2.LOAD.MISS_OUTSTANDING.CYCLES	Cycles while at least one load L2 cache miss in progress.
CACHE.L2.STORE.ACCESS.SPEC	L2 cache accesses initiated by store instructions. Speculatively executed instructions are also taken into account.
CACHE.L2.STORE.MISS.SPEC	L2 cache misses initiated by store instructions. Speculatively executed instructions are also taken into account.
CACHE.L2.STORE.HIT.SPEC	L2 cache hits initiated by store instructions. Speculatively executed instructions are also taken into account.
CACHE.L2.STORE.MERGE.SPEC	L2 cache hits initiated by store instructions where data is not yet in cache but was already requested by preceding miss. Speculatively executed instructions are also taken into account.
CACHE.L2.STORE.HIT.RFO.SPEC	L2 cache hits for store instructions with the purpose to get exclusive ownership. Speculatively executed instructions are also taken into account.
CACHE.L2.PREF.ISSUED	Prefetcher requests issued by L2 to next level cache or memory.
CACHE.L2.PREF.ACCESS	L2 cache accesses caused by prefetcher.
CACHE.L2.PREF.HIT	L2 cache hits caused by prefetcher.
CACHE.L2.PREF.MISS	L2 cache misses caused by prefetcher.
CACHE.L2.PREF.MERGE	L2 cache hits caused by prefetcher where data is not yet in cache but was already requested by preceding miss.
CACHE.L2.PREF.UNUSED	Number of cachelines brought into L2 by prefetcher and evicted without being accessed even once.
CACHE.L2.WB	Writebacks to next level cache or memory.
CACHE.SNOOP.LOCAL_REQ_REMOTE_HITM.SPEC	Private cache misses where data was found in another core cache in modified state. This event can be used to accout for contested accesses cases where several cores read/write the same cachelines.
CACHE.SNOOP.REMOTE_REQ_LOCAL_HITM	Snoop requests which found cacheline in the core cache in modified state. This event can be used to accout for contested accesses cases where several cores read/write the same cachelines.
CACHE.L3.LOAD.ACCESS.SPEC	L3 cache accesses for load instructions. Speculatively executed instructions are also taken into account.
CACHE.L3.LOAD.MISS.SPEC	L3 cache misses for load instructions. Speculatively executed instructions are also taken into account.
CACHE.L3.LOAD.HIT.SPEC	L3 cache hits for load instructions. Speculatively executed instructions are also taken into account.

Name	Description
CACHE.L3.LOAD.MERGE.SPEC	L3 cache hits for load instructions where data is not yet in cache but was already requested by preceding miss. Speculatively executed instructions are also taken into account.
CACHE.L3.LOAD.MISS_OUTSTANDING.CYCLES.SPEC	Cycles while at least one load L3 cache miss in progress.
CACHE.L3.STORE.ACCESS.SPEC	L3 cache accesses for store instructions. Speculatively executed instructions are also taken into account.
CACHE.L3.STORE.MISS.SPEC	L3 cache misses for store instructions. Speculatively executed instructions are also taken into account.
CACHE.L3.STORE.HIT.SPEC	L3 cache hits for store instructions. Speculatively executed instructions are also taken into account.
CACHE.L3.STORE.MERGE.SPEC	L3 cache hits for store instructions where data is not yet in cache but was already requested by preceding miss. Speculatively executed instructions are also taken into account.
CACHE.L3.STORE.HIT.RFO.SPEC	L3 cache hits for store instructions with the purpose to get exclusive ownership. Speculatively executed instructions are also taken into account.
CACHE.L3.PREF.ISSUED	Prefetcher requests issued by L3 to next level cache or memory.
CACHE.L3.PREF.ACCESS	L3 cache accesses caused by prefetcher.
CACHE.L3.PREF.HIT	L3 cache hits caused by prefetcher.
CACHE.L3.PREF.MISS	L3 cache misses caused by prefetcher.
CACHE.L3.PREF.MERGE	L3 cache hits caused by prefetcher where data is not yet in cache but was already requested by preceding miss.
CACHE.L3.PREF.UNUSED	Number of cachelines brought into L3 by prefetcher and evicted without being accessed even once.
CACHE.L3.WB	Writebacks to next level cache or memory.

Table 9. CACHE group events

Name	Description	Formula
CACHE.L1D.LOAD.SPEC.MPKI	The rate of speculative L1 data cache misses caused by data loads per kilo instructions retired	CACHE.L1D.LOAD.MISS.SPEC / INST.RET * 1000
CACHE.L1D.LOAD.MISS.SPEC.RATE	The ratio of speculative L1D cache misses to the total number of L1D accesses caused by data loads	CACHE.L1D.LOAD.MISS.SPEC / CACHE.L1D.LOAD.ACCESS.SPEC
CACHE.L1D.LOAD.MERGE.SPEC.PKI	The rate of speculative L1 data cache accesses which merged with previous cache miss per kilo instructions retired	CACHE.L1D.LOAD.MERGE.SPEC / INST.RET * 1000
CACHE.L1D.STORE.SPEC.MPKI	The rate of speculative L1 data cache misses caused by data stores per kilo instructions retired	CACHE.L1D.STORE.MISS.SPEC / INST.RET * 1000
CACHE.L1D.STORE.MISS.SPEC.RATE	The ratio of speculative L1D cache misses to the total number of L1D accesses caused by data stores	CACHE.L1D.STORE.MISS.SPEC / CACHE.L1D.STORE.ACCESS.SPEC
CACHE.L1D.PREF.ISSUED.PKI	The rate of prefetcher requests issued by L1D to next level cache per kilo instructions retired	CACHE.L1D.PREF.ISSUED / INST.RET * 1000
CACHE.L1D.PREF.UNUSED.RATE	The ratio of unused cachelines brought into L1D by prefetcher to the total number of prefetcher requests issued by L1D	CACHE.L1D.PREF.UNUSED / CACHE.L1D.PREF.ISSUED
CACHE.L1I.SPEC.MPKI	The rate of L1 instruction cache misses per kilo instructions retired	CACHE.L1I.MISS.SPEC / INST.RET * 1000
CACHE.L1I.MISS.SPEC.RATE	The ratio of L1 instruction cache misses to the total number of L1I accesses	CACHE.L1I.MISS.SPEC / CACHE.L1I.ACCESS.SPEC
CACHE.L1I.MERGE.SPEC.PKI	The rate of L1 instruction cache accesses which merged with previous cache miss per kilo instructions retired	CACHE.L1I.MERGE.SPEC / INST.RET * 1000
CACHE.L1I.MISS.IMPACT	The approximate ratio of cycles lost due to L1I misses	CACHE.L1I.MISS_OUTSTANDING.CYCLES / CYCLES.HART
CACHE.L2.LOAD.SPEC.MPKI	The rate of speculative L2 cache misses caused by data loads per kilo instructions retired	CACHE.L2.LOAD.MISS.SPEC / INST.RET * 1000
CACHE.L2.LOAD.MISS.SPEC.RATE	The ratio of speculative L2 cache misses to the total number of L2 accesses caused by data loads	CACHE.L2.LOAD.MISS.SPEC / CACHE.L2.LOAD.ACCESS.SPEC
CACHE.L2.STORE.SPEC.MPKI	The rate of speculative L2 cache misses caused by data stores per kilo instructions retired	CACHE.L2.STORE.MISS.SPEC / INST.RET * 1000
CACHE.L2.STORE.MISS.SPEC.RATE	The ratio of speculative L2 cache misses to the total number of L2 accesses caused by data stores	CACHE.L2.STORE.MISS.SPEC / CACHE.L2.STORE.ACCESS.SPEC
CACHE.L2.STORE.HIT.RFO.SPEC.PKI	The rate of L2 cache hits for store instructions with the purpose to get exclusive ownership per kilo instructions retired	CACHE.L2.STORE.HIT.RFO.SPEC / INST.RET * 1000
CACHE.L2.PREF.ISSUED.PKI	The rate of prefetcher requests issued by L2 to next level cache per kilo instructions retired	CACHE.L2.PREF.ISSUED / INST.RET * 1000
CACHE.L2.PREF.MPKI	The rate of L2 cache misses caused by prefetcher per kilo instructions retired	CACHE.L2.PREF.MISS / INST.RET * 1000
CACHE.L2.PREF.UNUSED.RATE	The ratio of unused cachelines brought into L2 by prefetcher to the total number of prefetcher requests issued by L2	CACHE.L2.PREF.UNUSED / CACHE.L2.PF.ISSUED

Name	Description	Formula
CACHE.L3.LOAD.SPEC.MPKI	The rate of speculative L3 cache misses caused by data loads per kilo instructions retired	CACHE.L3.LOAD.MISS.SPEC / INST.RET * 1000
CACHE.L3.LOAD.MISS.SPEC.RATE	The ratio of speculative L3 cache misses to the total number of L3 accesses caused by data loads	CACHE.L3.LOAD.MISS.SPEC / CACHE.L3.LOAD.ACCESS.SPEC
CACHE.L3.STORE.SPEC.MPKI	The rate of speculative L3 cache misses caused by data stores per kilo instructions retired	CACHE.L3.STORE.MISS.SPEC / INST.RET * 1000
CACHE.L3.STORE.MISS.SPEC.RATE	The ratio of speculative L3 cache misses to the total number of L3 accesses caused by data stores	CACHE.L3.STORE.MISS.SPEC / CACHE.L3.STORE.ACCESS.SPEC
CACHE.L3.PREF.ISSUED.PKI	The rate of prefetcher requests issued by L3 to next level cache per kilo instructions retired	CACHE.L3.PREF.ISSUED / INST.RET * 1000
CACHE.L3.PREF.MPKI	The rate of L3 cache misses caused by prefetcher per kilo instructions retired	CACHE.L3.PREF.MISS / INST.RET * 1000
CACHE.L3.PREF.UNUSED.RATE	The ratio of unused cachelines brought into L3 by prefetcher to the total number of prefetcher requests issued by L3	CACHE.L3.PREF.UNUSED / CACHE.L3.PREF.ISSUED

Table 10. CACHE group metrics

2.8. TLB (retirement)

This group contains events and metrics for data and instruction TLB caches (all levels) counted at retirement.

Name	Description
TLB.L1.LOAD.ACCESS.RET	Address translation requests for retired load instructions
TLB.L1.LOAD.MISS.RET	Address translation requests for retired load instructions which missed L1 TLB
TLB.L1.STORE.ACCESS.RET	Address translation requests for retired store instructions
TLB.L1.STORE.MISS.RET	Address translation requests for retired store instructions which missed L1 TLB
TLB.L2.LOAD.MISS.RET	Address translation requests for retired load instructions which missed L2 TLB
TLB.L2.STORE.MISS.RET	Address translation requests for retired store instructions which missed L2 TLB

Table 11. TLB group events

Name	Description	Formula
TLB.L1.LOAD.RET.MPKI	The rate of L1 TLB misses caused by data loads per kilo instructions retired	TLB.L1.LOAD.MISS.RET / INST.RET * 1000
TLB.L1.LOAD.RET.MISS.RATE	The ratio of L1 TLB load misses to the total number of L1 TLB load accesses retired	TLB.L1.LOAD.MISS.RET / TLB.L1.LOAD.ACCESS.RET
TLB.L1.STORE.RET.MPKI	The rate of L1 TLB misses caused by data stores per kilo instructions retired	TLB.L1.STORE.MISS.RET / INST.RET * 1000
TLB.L1.STORE.MISS.RET.RATE	The ratio of L1 TLB store misses to the total number of L1 TLB store accesses retired	TLB.L1.STORE.MISS.RET / TLB.L1.STORE.ACCESS.RET
TLB.L2.LOAD.RET.MPKI	The rate of L2 TLB misses caused by data loads per kilo instructions retired	TLB.L2.LOAD.MISS.RET / INST.RET * 1000
TLB.L2.LOAD.MISS.RET.RATE	The ratio of L2 TLB load misses to the total number of L2 TLB load accesses retired	TLB.L2.LOAD.MISS.RET / TLB.L2.LOAD.ACCESS.RET
TLB.L2.STORE.RET.MPKI	The rate of L2 TLB misses caused by data stores per kilo instructions retired	TLB.L2.STORE.MISS.RET / INST.RET * 1000
TLB.L2.STORE.MISS.RET.RATE	The ratio of L2 TLB store misses to the total number of L2 TLB store accesses retired	TLB.L2.STORE.MISS.RET / TLB.L2.STORE.ACCESS.RET

Table 12. TLB group metrics

2.9. TLB (speculative)

This group contains events and metrics for data and instruction TLB caches (all levels) counted speculatively.

Name	Description
TLB.L1.LOAD.ACCESS.SPEC	Address translation requests for load instructions. Speculatively executed instructions are also taken into account.
TLB.L1.CODE.ACCESS	Address translation requests for instructions fetch
TLB.L1.LOAD.MISS.SPEC	Address translation requests for load instructions which missed L1 TLB. Speculatively executed instructions are also taken into account.
TLB.L1.CODE.MISS.SPEC	Address translation requests for instructions fetch which missed L1 TLB
TLB.L1.LOAD.MISS_OUTSTANDING.CYCLES	Cycles while at least one load L1 TLB miss in progress.
TLB.L1.CODE.MISS_OUTSTANDING.CYCLES	Cycles while at least one L1 TLB miss for instructions fetch in progress
TLB.L1.STORE.ACCESS.SPEC	Address translation requests for store instructions. Speculatively executed instructions are also taken into account.
TLB.L1.STORE.MISS.SPEC	Address translation requests for store instructions which missed L1 TLB. Speculatively executed instructions are also taken into account.
TLB.L2.LOAD.MISS.SPEC	Address translation requests for load instructions which missed L2 TLB. Speculatively executed instructions are also taken into account.
TLB.L2.STORE.MISS.SPEC	Address translation requests for store instructions which missed L2 TLB. Speculatively executed instructions are also taken into account.
TLB.L2.CODE.MISS.SPEC	Address translation requests for instruction fetch which missed L2 TLB
TLB.L2.LOAD.MISS_OUTSTANDING.CYCLES	Cycles while at least one load L2 TLB miss in progress. Speculatively executed instructions are also taken into account.
TLB.L2.CODE.MISS_OUTSTANDING.CYCLES	Cycles while at least one L2 TLB miss for instructions fetch in progress

Table 13. TLB group events

Name	Description	Formula
TLB.L1.LOAD.SPEC.MPKI	The rate of L1 TLB misses caused by data loads per kilo instructions retired	TLB.L1.LOAD.MISS.SPEC / INST.RET * 1000
TLB.L1.LOAD.MISS.SPEC.RATE	The ratio of L1 TLB load misses to the total number of L1 TLB load accesses	TLB.L1.LOAD.MISS.SPEC / TLB.L1.LOAD.ACCESS.SPEC
TLB.L1.STORE.SPEC.MPKI	The rate of L1 TLB misses caused by data stores per kilo instructions retired	TLB.L1.STORE.MISS.SPEC / INST.RET * 1000
TLB.L1.STORE.MISS.SPEC.RATE	The ratio of L1 TLB store misses to the total number of L1 TLB store accesses	TLB.L1.STORE.MISS.SPEC / TLB.L1.STORE.ACCESS.SPEC
TLB.L1.CODE.SPEC.MPKI	The rate of L1 TLB misses caused by instruction fetches per kilo instructions retired	TLB.L1.CODE.MISS.SPEC / INST.RET * 1000
TLB.L1.CODE.MISS.SPEC.RATE	The ratio of L1 TLB instruction fetch misses to the total number of L1 TLB instruction fetch accesses	TLB.L1.CODE.MISS.SPEC / TLB.L1.CODE.ACCESS
TLB.L2.LOAD.SPEC.MPKI	The rate of L2 TLB misses caused by data loads per kilo instructions retired	TLB.L2.LOAD.MISS.SPEC / INST.RET * 1000
TLB.L2.LOAD.MISS.SPEC.RATE	The ratio of L2 TLB load misses to the total number of L2 TLB load accesses	TLB.L2.LOAD.MISS.SPEC / TLB.L2.LOAD.ACCESS.SPEC
TLB.L2.STORE.SPEC.MPKI	The rate of L2 TLB misses caused by data stores per kilo instructions retired	TLB.L2.STORE.MISS.SPEC / INST.RET * 1000
TLB.L2.STORE.MISS.SPEC.RATE	The ratio of L2 TLB store misses to the total number of L2 TLB store accesses	TLB.L2.STORE.MISS.SPEC / TLB.L2.STORE.ACCESS
TLB.L2.CODE.SPEC.MPKI	The rate of L2 TLB misses caused by instruction fetches per kilo instructions retired	TLB.L2.CODE.MISS.SPEC / INST.RET * 1000
TLB.L2.CODE.MISS.SPEC.RATE	The ratio of L2 TLB instruction fetch misses to the total number of L2 TLB instruction fetch accesses	TLB.L2.CODE.MISS.SPEC / TLB.L2.CODE.ACCESS.SPEC
TLB.L1.MISS.IMPACT	The approximate ratio of cycles lost due to TLB (all levels) missed by load instructions	TLB.L1.LOAD.MISS_OUTSTANDING.CYCLES / CYCLES.HART
TLB.L2.MISS.IMPACT	The approximate ratio of cycles lost due to L2 TLB missed by load instructions	TLB.L2.LOAD.MISS_OUTSTANDING.CYCLES / CYCLES.HART
TLB.L2.HIT.IMPACT	The approximate ratio of cycles lost due to L1 TLB missed and L2 TLB hit by load instructions	(TLB.L1.LOAD.MISS_OUTSTANDING.CYCLES - TLB.L2.LOAD.MISS_OUTSTANDING.CYCLES) / CYCLES.HART

Table 14. TLB group metrics

2.10. TOPDOWN

This group contains events and metrics related for Topdown Microarchitecture Analysis (TMA)

methodology.

The TMA methodology categorizes CPU execution time at a high level first. This step flags (reports high fraction value) some domain(s) for possible investigation. Next, the user can drill down into those flagged domains, and can safely ignore all non-flagged domains. The process is repeated in a hierarchical manner until a specific performance issue is determined or at least a small subset of candidate issues is identified for potential investigation.

Given the highly sophisticated microarchitecture, the first interesting question is how and where to do the first level breakdown? TMA chooses the issue point as it is the natural border that splits the frontend and backend portions of machine.

At issue point it classifies each pipeline-slot into one of four base categories: Frontend Bound, Backend Bound, Bad Speculation and Retiring. If a uop is issued in a given cycle, it would eventually either get retired or cancelled. Thus it can be attributed to either Retiring or Bad Speculation respectively. Otherwise it can be split into whether there was a backend-stall or not. A backend-stall is a backpressure mechanism the Backend asserts upon resource unavailability (e.g. lack of load buffer entries). In such a case TMA attributes the stall to the Backend, since even if the Frontend was ready with more uops it would not be able to pass them down the pipeline. If there was no backend-stall, it means the Frontend should have delivered some uops while the Backend was ready to accept them; hence it is tagged with Frontend Bound.

Name	Description
TOPDOWN.SLOTS	TMA slots available for an unhalted hart. In case of SMT the counts are distributed cycles between all active harts in the same physical core.
TOPDOWN.FRONTEND_BOUND.SLOTS	TMA slots unused due to the frontend did not supply enough operations
TOPDOWN.BAD_SPECULATION.SLOTS	TMA slots wasted due to incorrect speculations. This include slots used to issue uops that do not eventually get retired and slots for which the issue-pipeline was blocked due to recovery from earlier incorrect speculation. For example; wasted slots due to miss-predicted branches should be accounted by this event. Incorrect data speculation followed by Memory Ordering Nukes is another example.
TOPDOWN.BAD_SPECULATION.CONTROL_FLOW.SLOTS	TMA slots wasted due to incorrect control flow speculations.
TOPDOWN.BAD_SPECULATION.MEM_ORDERING.SLOTS	TMA slots wasted due to memory ordering violations.
TOPDOWN.BACKEND_BOUND.SLOTS	TMA slots unused due to the lack of backend resources
TOPDOWN.BACKEND_BOUND.MEMORY.SLOTS	TMA slots unused due to the stalls caused by load and store instructions
TOPDOWN.BACKEND_BOUND.CORE.SLOTS	TMA slots unused due to the non-memory stalls
TOPDOWN.BACKEND_BOUND.CORE.SERIALIZING.SLOTS	Number of slots wasted due to serializing operations like fence or csr accesses
TOPDOWN.BACKEND_BOUND.MEMORY.ADDR.SLOTS	TMA slots wasted while waiting for address generation and translation
TOPDOWN.BACKEND_BOUND.MEMORY.ADDR.TLB.L1_MISS.SLOTS	TMA slots wasted while waiting for address translation which missed L1 TLB
TOPDOWN.BACKEND_BOUND.MEMORY.ADDR.TLB.L2_MISS.SLOTS	TMA slots wasted while waiting for address translation which missed L2 TLB
TOPDOWN.BACKEND_BOUND.MEMORY.DATA.SLOTS	TMA slots wasted while waiting for data
TOPDOWN.BACKEND_BOUND.MEMORY.DATA.L1_MISS.SLOTS	TMA slots unused due to the stalls caused by memory instructions which missed L1 data cache
TOPDOWN.BACKEND_BOUND.MEMORY.DATA.L2_MISS.SLOTS	TMA slots unused due to the stalls caused by memory instructions which missed L2 cache
TOPDOWN.BACKEND_BOUND.MEMORY.DATA.L3_MISS.SLOTS	TMA slots unused due to the stalls caused by memory instructions which missed L3 cache

Table 15. TOPDOWN group events

Name	Description	Formula
TOPDOWN.SLOTS	For implementations which do not provide TOPDOWN.SLOTS as explicit event it is possible to calculate it as metric using CYCLES.HART event.	pipeline_width * CYCLES.HART
TOPDOWN.BAD_SPECULATION.SLOTS	For implementations which do not provide TOPDOWN.BAD_SPECULATION.SLOTS as explicit event it is possible to calculate it as metric. It consiste of two part - the number of cancelled operations (UOP.SPEC - UOP.RET) and the number of slots wasted on recovery from pipeline flush (pipeline_width * PRD.PIPELINE_FLUSH.RECOVERY_CYCLES)	UOP.SPEC - UOP.RET + pipeline_width * PRD.PIPELINE_FLUSH.RECOVERY_CYCLES
TOPDOWN.BACKEND_BOUND.CORE.SLOTS	For implementations which do not provide TOPDOWN.BACKEND_BOUND.CORE.SLOTS as explicit event it is possible to calculate it as metric by subtracting memory bound slots from all backend bound slots	TOPDOWN.BACKEND_BOUND.SLOTS - TOPDOWN.BACKEND_BOUND.MEMORY.SLOTS
TOPDOWN.BAD_SPECULATION	Fraction of slots wasted due to incorrect speculations. This include slots used to issue uops that do not eventually get retired and slots for which the issue-pipeline was blocked due to recovery from earlier incorrect speculation.	TOPDOWN.BAD_SPECULATION.SLOTS / TOPDOWN.SLOTS
TOPDOWN.BAD_SPECULATION.MISPRED.RATE	Fraction of slots wasted due to incorrect control flow speculations	TOPDOWN.BAD_SPECULATION.MISPRED.SLOTS / TOPDOWN.SLOTS
TOPDOWN.BAD_SPECULATION.MEM_ORDERING.RATE	Fraction of slots wasted due to memory ordering violations	TOPDOWN.BAD_SPECULATION.MEM_ORDERING.SLOTS / TOPDOWN.SLOTS
TOPDOWN.BAD_SPECULATION.OTHER.RATE	Fraction of slots wasted due to reasons other than control flow mis-speculations or memory ordering violations	TOPDOWN.BAD_SPECULATION.MEM_ORDERING.SLOTS / TOPDOWN.SLOTS
TOPDOWN.FRONTEND_BOUND.RATE	Fraction of slots unused due to the frontend did not supply enough operations. Frontend Bound denotes unutilized slots when there is no Backend stall - i.e. when Frontend delivered no uops while Backend could have accepted them. For example, stalls due to instruction cache misses would be categorized under Frontend Bound.	TOPDOWN.FRONTEND_BOUND.SLOTS / TOPDOWN.SLOTS
TOPDOWN.RETIRING.RATE	Fraction of slots utilized by useful work i.e. issued uops that eventually get retired. Ideally all pipeline slots would be attributed to the Retiring category. Retiring of 100% would indicate the maximum Pipeline_Width throughput was achieved. Maximizing Retiring typically increases the Instructions-per-cycle (IPC).	UOP.RET / TOPDOWN.SLOTS
TOPDOWN.BACKEND_BOUND.RATE	Fraction of slots unused due to the due to lack of backend resources. Backend Bound denotes unutilized slots due to a lack of required resources for accepting new uops in the Backend.	TOPDOWN.BACKEND_BOUND.SLOTS / TOPDOWN.SLOTS
TOPDOWN.BACKEND_BOUND.MEMORY_BOUND.RATE	Fraction of slots unused due to the memory subsystem stalls inside the backend. Memory Bound estimates fraction of slots where pipeline is likely stalled due to demand load or store instructions. This accounts mainly for (1) non-completed in-flight memory demand loads which coincides with execution units starvation; in addition to (2) cases where stores could impose backpressure on the pipeline when many of them get buffered at the same time (less common out of the two).	TOPDOWN.BACKEND_BOUND.MEMORY.SLOTS / TOPDOWN.SLOTS
TOPDOWN.BACKEND_BOUND.CORE_BOUND.RATE	Fraction of slots unused due to the non-memory stalls inside the backend. Shortage in hardware compute resources or dependencies in software instructions are both categorized under Core Bound. Hence it may indicate the machine ran out of an out-of-order resource; certain execution units are overloaded or dependencies in program's data- or instruction-flow are limiting the performance (e.g. chained long-latency arithmetic operations).	TOPDOWN.BACKEND_BOUND.CORE.SLOTS / TOPDOWN.SLOTS
TOPDOWN.BACKEND_BOUND.CORE.SERIALIZING.RATE	Fraction of slots unused due to serializing operations like fence or csr accesses.	TOPDOWN.BACKEND_BOUND.CORE.SERIALIZING.SLOTS / TOPDOWN.SLOTS
TOPDOWN.BACKEND_BOUND.MEMORY_BOUND.ADDR_BOUND.RATE	Fraction of slots wasted while waiting for address generation and translation	TOPDOWN.BACKEND_BOUND.MEMORY.ADDR.SLOTS / TOPDOWN.SLOTS
TOPDOWN.BACKEND_BOUND.MEMORY_BOUND.ADDR_BOUND.TLB_L1_BOUND.RATE	Fraction of slots wasted while waiting for address generation without missing L1 TLB	(TOPDOWN.BACKEND_BOUND.MEMORY.ADDR.SLOTS - TOPDOWN.BACKEND_BOUND.MEMORY.ADDR.TLB.L1_MISS.SLOTS) / TOPDOWN.SLOTS
TOPDOWN.BACKEND_BOUND.MEMORY_BOUND.ADDR_BOUND.TLB_L2_BOUND.RATE	Fraction of slots wasted while waiting for address generation which hit L2 TLB	(TOPDOWN.BACKEND_BOUND.MEMORY.ADDR.TLB.L1_MISS.SLOTS - TOPDOWN.BACKEND_BOUND.MEMORY.ADDR.TLB.L2_MISS.SLOTS) / TOPDOWN.SLOTS
TOPDOWN.BACKEND_BOUND.MEMORY_BOUND.ADDR_BOUND.PAGE_WALK_BOUND.RATE	Fraction of slots wasted while waiting for address translation which needed page walk (missed all TLB levels)	TOPDOWN.BACKEND_BOUND.MEMORY.ADDR.TLB.L2_MISS.SLOTS / TOPDOWN.SLOTS

Name	Description	Formula
TOPDOWN.BACKEND_BOUND.MEMORY_BOUND.DATA_BOUND.RATE	Fraction of slots wasted while waiting for data	TOPDOWN.BACKEND_BOUND.MEMORY.DATA.SLOTS / TOPDOWN.SLOTS
TOPDOWN.BACKEND_BOUND.MEMORY_BOUND.DATA_BOUND.L1_BOUND.RATE	Fraction of slots unused due to the stalls caused by load instructions which got data from L1 data cache	(TOPDOWN.BACKEND_BOUND.MEMORY.DATA.SLOTS - TOPDOWN.BACKEND_BOUND.MEMORY.DATA.L1_MISS.SLOTS) / TOPDOWN.SLOTS
TOPDOWN.BACKEND_BOUND.MEMORY_BOUND.DATA_BOUND.L2_BOUND.RATE	Fraction of slots unused due to the stalls caused by load instructions which got data from L2 cache	(TOPDOWN.BACKEND_BOUND.MEMORY.DATA.L1_MISS.SLOTS - TOPDOWN.BACKEND_BOUND.MEMORY.DATA.DATA.DATA.DATA.DATA.DATA.DATA.DATA.DATA.DATA.DATA.DATA.DATA.DATA.DATA.DATA.DATA.DATA.L2_MISS.SLOTS) / TOPDOWN.SLOTS
TOPDOWN.BACKEND_BOUND.MEMORY_BOUND.DATA_BOUND.L3_BOUND.RATE	Fraction of slots unused due to the stalls caused by load instructions which got data from L3 cache	(TOPDOWN.BACKEND_BOUND.MEMORY.DATA.L2_MISS.SLOTS - TOPDOWN.BACKEND_BOUND.MEMORY.DATA.L3_MISS.SLOTS) / TOPDOWN.SLOTS
TOPDOWN.BACKEND_BOUND.MEMORY_BOUND.DATA_BOUND.EXTERNAL_MEM_BOUND.RATE	Fraction of slots unused due to the stalls caused by load instructions which got data from external memory	TOPDOWN.BACKEND_BOUND.MEMORY.DATA.L3_MISS.SLOTS / TOPDOWN.SLOTS

Table 16. TOPDOWN group metrics

2.11. RVV (retirement)

This group contains events and metrics related to vectorized operations counted at retirement.

Name	Description
INST.RVV.RET	Number of RVV instructions retired
INST.RVV.INT.RET	Number of integer RVV instructions retired
INST.RVV.FP.RET	Number of floating point RVV instructions retired
RVV.ELEMENT.UNMASKED.INT8.RET	Number of 8-bit integer element operation retired. For example, if we have SEW=8, LMUL=1, VLEN=128 and doing vector integer arith instruction - it should increment the RVV.ELEMENT.UNMASKED.INT8 counter by 16. Masked-out elements should not increment the counter - so in the previous example if half of the lanes are masked the RVV.ELEMENT.UNMASKED.INT8 will be incremented by 8. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.
RVV.ELEMENT.INT8.RET	Number of 8-bit integer element operation retired not taking into account masking. For example, if we have SEW=8, LMUL=1, VLEN=128 and doing vector integer arith instruction - it should increment the RVV.ELEMENT.INT8 counter by 16. Mask should not be taken into account - so in the previous example if half of the lanes are masked the RVV.ELEMENT.INT8 will still be incremented by 16. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.
RVV.ELEMENT.UNMASKED.INT16.RET	Number of 16-bit integer element operation retired. For example, if we have SEW=16, LMUL=1, VLEN=128 and doing vector integer arith instruction - it should increment the RVV.ELEMENT.UNMASKED.INT16 counter by 8. Masked-out elements should not increment the counter - so in the previous example half of the lanes are masked the RVV.ELEMENT.UNMASKED.INT16 counter will be incremented by 4. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.
RVV.ELEMENT.INT16.RET	Number of 16-bit integer element operation retired not taking into account masking. For example, if we have SEW=16, LMUL=1, VLEN=128 and doing vector integer arith instruction - it should increment the RVV.ELEMENT.INT16 counter by 8. Mask should not be taken into account - so in the previous example if half of the lanes are masked the RVV.ELEMENT.INT16 counter will still be incremented by 8. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.
RVV.ELEMENT.UNMASKED.INT32.RET	Number of 32-bit integer element operation retired. For example, if we have SEW=32, LMUL=1, VLEN=128 and doing vector integer arith instruction - it should increment the RVV.ELEMENT.UNMASKED.INT16 counter by 4. Masked-out elements should not increment the counter - so in the previous example if half of the lanes are masked the RVV.ELEMENT.UNMASKED.INT32 counter will be incremented by 2. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.
RVV.ELEMENT.INT32.RET	Number of 32-bit integer element operation retired not taking into account masking. For example, if we have SEW=32, LMUL=1, VLEN=128 and doing vector integer arith instruction - it should increment the RVV.ELEMENT.INT16 counter by 4. Mask should not be taken into account - so in the previous example if half of the lanes are masked the RVV.ELEMENT.INT32 counter will still be incremented by 4. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.
RVV.ELEMENT.UNMASKED.INT64.RET	Number of 64-bit integer element operation retired. For example, if we have SEW=64, LMUL=1, VLEN=128 and doing vector integer arith instruction - it should increment the RVV.ELEMENT.UNMASKED.INT64 counter by 2. Masked-out elements should not increment the counter - so in the previous example half of the lanes are masked the RVV.ELEMENT.UNMASKED.INT64 counter will be incremented by 1. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.

Name	Description
RVV.ELEMENT.INT64.RET	Number of 64-bit integer element operation retired not taking into account masking. For example, if we have SEW=64, LMUL=1, VLEN=128 and doing vector integer arith instruction - it should increment the RVV.ELEMENT.INT64 counter by 2. Mask should not be taken into account - so in the previous example if half of the lanes are masked the RVV.ELEMENT.INT64 counter will still be incremented by 2. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.
RVV.ELEMENT.UNMASKED.FP_SINGLE.RET	Number of single-precision floating point element operation retired. For example, if we have SEW=32, LMUL=1, VLEN=128 and doing vector FP arith instruction - it should increment the RVV.ELEMENT.UNMASKED.FP_SINGLE counter by 4. Masked-out elements should not increment the counter - so in the previous example if half of the lanes are masked the RVV.ELEMENT.UNMASKED.FP_SINGLE counter will be incremented by 2. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.
RVV.ELEMENT.FP_SINGLE.RET	Number of single-precision floating point element operation retired not taking into account masking. For example, if we have SEW=32, LMUL=1, VLEN=128 and doing vector FP arith instruction - it should increment the RVV.ELEMENT.FP_SINGLE counter by 4. Mask should not be taken into account - so in the previous example if half of the lanes are masked the RVV.ELEMENT.FP_SINGLE counter will still be incremented by 4. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.
RVV.ELEMENT.UNMASKED.FP_DOUBLE.RET	Number of double-precision floating point element operation retired. For example, if we have SEW=64, LMUL=1, VLEN=128 and doing vector FP arith instruction - it should increment the RVV.ELEMENT.UNMASKED.FP_DOUBLE counter by 2. Masked-out elements should not increment the counter - so in the previous example if half of the lanes are masked the RVV.ELEMENT.UNMASKED.FP_DOUBLE counter will be incremented by 1. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.
RVV.ELEMENT.FP_DOUBLE.RET	Number of double-precision floating point element operation retired not taking into account masking. For example, if we have SEW=64, LMUL=1, VLEN=128 and doing vector FP arith instruction - it should increment the RVV.ELEMENT.FP_DOUBLE counter by 2. Mask should not be taken into account - so in the previous example if half of the lanes are masked the RVV.ELEMENT.FP_DOUBLE counter will still be incremented by 2. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.

Table 17. RVV group events

Name	Description	Formula
RVV.FLOPC_SINGLE.RET	Vector single-precision floating point operations retired per cycle	RVV.ELEMENT.UNMASKED.FP_SINGLE.RET / CYCLES.HART
RVV.FLOPC_DOUBLE.RET	Vector double-precision floating point operations retired per cycle	RVV.ELEMENT.UNMASKED.FP_DOUBLE.RET / CYCLES.HART
RVV.FLOP.RET	Vector floating point operations retired	RVV.ELEMENT.UNMASKED.FP_SINGLE.RET + RVV.ELEMENT.UNMASKED.FP_DOUBLE.RET
RVV.FLOPC.RET	Vector floating point operations retired per cycle	RVV.FLOP.RET / CYCLES.HART
RVV.GFLOP.RET	Vector giga floating point operations retired	RVV.FLOP.RET / 1000000000.0
RVV.TFLOP.RET	Vector tera floating point operations retired	RVV.FLOP.RET / 1000000000000.0
RVV.IOPC_8.RET	Vector 8-bits integer operations retired per cycle	RVV.ELEMENT.UNMASKED.INT8.RET / CYCLES.HART
RVV.IOPC_16.RET	Vector 16-bits integer operations retired per cycle	RVV.ELEMENT.UNMASKED.INT16.RET / CYCLES.HART
RVV.IOPC_32.RET	Vector 32-bits integer operations retired per cycle	RVV.ELEMENT.UNMASKED.INT32.RET / CYCLES.HART
RVV.IOPC_64.RET	Vector 64-bits integer operations retired per cycle	RVV.ELEMENT.UNMASKED.INT64.RET / CYCLES.HART
RVV.IOP.RET	Vector integer operations retired	RVV.ELEMENT.UNMASKED.INT8.RET + RVV.ELEMENT.UNMASKED.INT16.RET + RVV.ELEMENT.UNMASKED.INT32.RET + RVV.ELEMENT.UNMASKED.INT64.RET
RVV.IOPC.RET	Vector integer operations retired per cycle	RVV.IOP.RET / CYCLES.HART
RVV.TIOP.RET	Vector tera integer operations retired	RVV.IOP.RET / 1000000000000
RVV.TOP.RET	Vector tera operations retired	RVV.TFLOP.RET + RVV.TIOP.RET

Table 18. RVV group metrics

2.12. RVV (speculative)

This group contains events and metrics related to vectorized operations counted speculatively.

Name	Description
INST.RVV.SPEC	Number of RVV instructions executed
INST.RVV.INT.SPEC	Number of integer RVV instructions executed
INST.RVV.FP.SPEC	Number of floating point RVV instructions executed
RVV.ELEMENT.UNMASKED.INT8.SPEC	Number of 8-bit integer element operation executed. For example, if we have SEW=8, LMUL=1, VLEN=128 and doing vector integer arith instruction - it should increment the RVV.ELEMENT.UNMASKED.INT8 counter by 16. Masked-out elements should not increment the counter - so in the previous example if half of the lanes are masked the RVV.ELEMENT.UNMASKED.INT8 will be incremented by 8. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.
RVV.ELEMENT.INT8.SPEC	Number of 8-bit integer element operation executed. For example, if we have SEW=8, LMUL=1, VLEN=128 and doing vector integer arith instruction - it should increment the RVV.ELEMENT.INT8 counter by 16. Mask should not be taken into account - so in the previous example if half of the lanes are masked the RVV.ELEMENT.INT8 will still be incremented by 16. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.
RVV.ELEMENT.UNMASKED.INT16.SPEC	Number of 16-bit integer element operation executed. For example, if we have SEW=16, LMUL=1, VLEN=128 and doing vector integer arith instruction - it should increment the RVV.ELEMENT.UNMASKED.INT16 counter by 8. Masked-out elements should not increment the counter - so in the previous example half of the lanes are masked the RVV.ELEMENT.UNMASKED.INT16 counter will be incremented by 4. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.
RVV.ELEMENT.INT16.SPEC	Number of 16-bit integer element operation executed. For example, if we have SEW=16, LMUL=1, VLEN=128 and doing vector integer arith instruction - it should increment the RVV.ELEMENT.INT16 counter by 8. Mask should not be taken into account - so in the previous example if half of the lanes are masked the RVV.ELEMENT.INT16 counter will still be incremented by 8. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.
RVV.ELEMENT.UNMASKED.INT32.SPEC	Number of 32-bit integer element operation executed. For example, if we have SEW=32, LMUL=1, VLEN=128 and doing vector integer arith instruction - it should increment the RVV.ELEMENT.UNMASKED.INT16 counter by 4. Masked-out elements should not increment the counter - so in the previous example if half of the lanes are masked the RVV.ELEMENT.UNMASKED.INT32 counter will be incremented by 2. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.
RVV.ELEMENT.INT32.SPEC	Number of 32-bit integer element operation executed. For example, if we have SEW=32, LMUL=1, VLEN=128 and doing vector integer arith instruction - it should increment the RVV.ELEMENT.INT16 counter by 4. Mask should not be taken into account - so in the previous example if half of the lanes are masked the RVV.ELEMENT.INT32 counter will still be incremented by 4. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.
RVV.ELEMENT.UNMASKED.INT64.SPEC	Number of 64-bit integer element operation executed. For example, if we have SEW=64, LMUL=1, VLEN=128 and doing vector integer arith instruction - it should increment the RVV.ELEMENT.UNMASKED.INT64 counter by 2. Masked-out elements should not increment the counter - so in the previous example half of the lanes are masked the RVV.ELEMENT.UNMASKED.INT64 counter will be incremented by 1. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.
RVV.ELEMENT.INT64.SPEC	Number of 64-bit integer element operation executed. For example, if we have SEW=64, LMUL=1, VLEN=128 and doing vector integer arith instruction - it should increment the RVV.ELEMENT.INT64 counter by 2. Mask should not be taken into account - so in the previous example if half of the lanes are masked the RVV.ELEMENT.INT64 counter will still be incremented by 2. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.
RVV.ELEMENT.UNMASKED.FP_SINGLE.SPEC	Number of single-precision floating point element operation executed. For example, if we have SEW=32, LMUL=1, VLEN=128 and doing vector FP arith instruction - it should increment the RVV.ELEMENT.UNMASKED.FP_SINGLE counter by 4. Masked-out elements should not increment the counter - so in the previous example if half of the lanes are masked the RVV.ELEMENT.UNMASKED.FP_SINGLE counter will be incremented by 2. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.
RVV.ELEMENT.FP_SINGLE.SPEC	Number of single-precision floating point element operation executed. For example, if we have SEW=32, LMUL=1, VLEN=128 and doing vector FP arith instruction - it should increment the RVV.ELEMENT.FP_SINGLE counter by 4. Mask should not be taken into account - so in the previous example if half of the lanes are masked the RVV.ELEMENT.FP_SINGLE counter will still be incremented by 4. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.
RVV.ELEMENT.UNMASKED.FP_DOUBLE.SPEC	Number of double-precision floating point element operation executed. For example, if we have SEW=64, LMUL=1, VLEN=128 and doing vector FP arith instruction - it should increment the RVV.ELEMENT.UNMASKED.FP_DOUBLE counter by 2. Masked-out elements should not increment the counter - so in the previous example if half of the lanes are masked the RVV.ELEMENT.UNMASKED.FP_DOUBLE counter will be incremented by 1. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.
RVV.ELEMENT.FP_DOUBLE.SPEC	Number of double-precision floating point element operation executed. For example, if we have SEW=64, LMUL=1, VLEN=128 and doing vector FP arith instruction - it should increment the RVV.ELEMENT.FP_DOUBLE counter by 2. Mask should not be taken into account - so in the previous example if half of the lanes are masked the RVV.ELEMENT.FP_DOUBLE counter will still be incremented by 2. For multiply-add instructions each element operation should increment counter by 2 to account for both multiplication and addition.

Table 19. RVV group events

Name	Description	Formula
RVV.FLOPC_SINGLE.SPEC	Vector single-precision floating point operations executed per cycle	RVV.ELEMENT.UNMASKED.FP_SINGLE.SPEC / CYCLES.HART

Name	Description	Formula
RVV.FLOPC_DOUBLE.SPEC	Vector double-precision floating point operations executed per cycle	RVV.ELEMENT.UNMASKED.FP_DOUBLE.SPEC / CYCLES.HART
RVV.FLOP.SPEC	Vector floating point operations executed	RVV.ELEMENT.UNMASKED.FP_SINGLE.SPEC + RVV.ELEMENT.UNMASKED.FP_DOUBLE.SPEC
RVV.FLOPC.SPEC	Vector floating point operations executed per cycle	RVV.FLOP.SPEC / CYCLES.HART.SPEC
RVV.GFLOP.SPEC	Vector giga floating point operations executed	RVV.FLOP.SPEC / 1000000000.0
RVV.TFLOP.SPEC	Vector tera floating point operations executed	RVV.FLOP.SPEC / 1000000000000.0
RVV.IOPC_8.SPEC	Vector 8-bits integer operations executed per cycle	RVV.ELEMENT.UNMASKED.INT8.SPEC / CYCLES.HART
RVV.IOPC_16.SPEC	Vector 16-bits integer operations executed per cycle	RVV.ELEMENT.UNMASKED.INT16.SPEC / CYCLES.HART
RVV.IOPC_32.SPEC	Vector 32-bits integer operations executed per cycle	RVV.ELEMENT.UNMASKED.INT32.SPEC / CYCLES.HART
RVV.IOPC_64.SPEC	Vector 64-bits integer operations executed per cycle	RVV.ELEMENT.UNMASKED.INT64.SPEC / CYCLES.HART
RVV.IOP.SPEC	Vector integer operations executed	RVV.ELEMENT.UNMASKED.INT8.SPEC + RVV.ELEMENT.UNMASKED.INT16.SPEC + RVV.ELEMENT.UNMASKED.INT32.SPEC + RVV.ELEMENT.UNMASKED.INT64.SPEC
RVV.IOPC.SPEC	Vector integer operations executed per cycle	RVV.IOP.SPEC / CYCLES.HART
RVV.TIOP.SPEC	Vector tera integer operations executed	RVV.IOP.SPEC / 1000000000000
RVV.TOP.SPEC	Vector tera operations executed	RVV.TFLOP.SPEC + RVV.TIOP.SPEC

Table 20. RVV group metrics